

[On This Page](#) >

coredns

Clan-internal DNS and service exposure

Network

❗ Experimental

This service is experimental and will change in the future.

This module enables hosting clan-internal services easily, which can be resolved inside your VPN. This allows defining a custom top-level domain (e.g. `.clan`) and exposing endpoints from a machine to others, which will be accessible under `http://<service>.clan` in your browser.

The service consists of two roles:

- A `server` role: This is the DNS-server that will be queried when trying to resolve clan-internal services. It defines the top-level domain.
- A `default` role: This does two things. First, it sets up the nameservers so that clan-internal queries are resolved via the `server` machine, while external queries are resolved as normal via DHCP. Second, it allows exposing services (see example below).

Example Usage

Here the machine `dnsserver` is designated as internal DNS-server for the TLD `.foo`. `server01` will host an application that shall be reachable at `http://one.foo` and `server02` is going to be reachable at `http://two.foo`. `client` is any other machine that is part of the clan but does not host any services.

When `client` tries to resolve `http://one.foo`, the DNS query will be routed to `dnsserver`, which will answer with `192.168.1.3`. If it tries to resolve some external domain (e.g. `https://clan.lol`), the query will not be routed to `dnsserver` but resolved as before, via the nameservers advertised by DHCP.

```
1 inventory = {
2
3   machines = {
4     dnsserver = { }; # 192.168.1.2
5     server01 = { }; # 192.168.1.3
6     server02 = { }; # 192.168.1.4
7     client = { }; # 192.168.1.5
8   };
9
10  instances = {
11    coredns = {
12
13      module.name = "@clan/coredns";
14      module.input = "self";
15
16      # Add the default role to all machines, including `client`
17      roles.default.tags.all = { };
18
19      # DNS server queries to http://<name>.foo are resolved here
20      roles.server.machines."dnsserver".settings = {
21        ip = "192.168.1.2";
22        tld = "foo";
23      };
24
25      # First service
26      # Registers http://one.foo will resolve to 192.168.1.3
27      # underlying service runs on server01
28      roles.default.machines."server01".settings = {
29        ip = "192.168.1.3";
30        services = [ "one" ];
31      };
32
33      # Second service
34      roles.default.machines."server02".settings = {
35        ip = "192.168.1.4";
36        services = [ "two" ];
37      };
38    };
39  };
40 };
```

Roles

The coredns service has the following roles:

- default
- server

Options for the `default` role

dnsPort

Port of the clan-internal DNS server

Type: `signed integer`

Default:

1053

Declared in: [clanServices/coredns/default.nix](#)

ip

IP on which the services will listen

Type: `string`

Declared in: [clanServices/coredns/default.nix](#)

services

Service endpoints this host exposes (without TLD). Each entry will be resolved to `<entry>.<tld>` using the configured top-level domain.

Type: `list of string`

Default:

[]

Declared in: [clanServices/coredns/default.nix](#)

Options for the `server` role

dnsPort

Port of the clan-internal DNS server

Type: `signed integer`

Default:

1053

Declared in: [clanServices/coredns/default.nix](#)

ip

IP for the DNS to listen on

Type: `string`

Declared in: [clanServices/coredns/default.nix](#)

tld

Top-level domain for this instance. All services below this will be resolved internally.

Type: `string`

Default:

"clan"

Declared in: [clanServices/coredns/default.nix](#)

← Previous
certificates

Next →
data-mesher

Need help? Reach out to [our community](#).